

Reviewer Pack — Free Cumulant Gap in Read-Twice BPs for IP_2

Entry cc2d7e2bed97 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 19:40:46 UTC

Free Cumulant Gap in Read-Twice BPs for IP_2

Entry ID: cc2d7e2bed97 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 19:40:46 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For read-twice BPs computing IP_2, the free cumulant $\rho(P)$ of the transition matrix satisfies $\rho(P) = \Omega(n)$ for the trivial BP (constant 0), but $\rho(P) = O(\log \text{size}(P))$ for all other BP representations. The free cumulant is defined as the sum of non-crossing partitions' moments in the Lévy metric.

Rationale (proposer's reasoning):

Free cumulants capture non-commutative dependencies in tensor structures, which may expose structural differences between trivial and complex BPs. The Lévy metric's sensitivity to tail behavior could amplify the gap between constant and exponential size BPs.

Taxonomy category: BP_READTWICE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cad22781e79aec85

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def factorial(n):
        if n == 0 or n == 1:
            return 1
        result = 1
        for i in range(2, n + 1):
            result *= i
        return result

    def binomial_coefficient(n, k):
        return factorial(n) // (factorial(k) * factorial(n - k))

    def non_crossing_partitions(n):
        if n == 0:
            return [[]]
        partitions = []
        for k in range(1, n + 1):
            for partition in non_crossing_partitions(k - 1):
                new_partition = partition[:]
                new_partition.append([k] + [x + 1 for x in partition[-1]])
                partitions.append(new_partition)
            for i in range(len(partition) - 1):
                new_partition = partition[:]
                new_partition[i].append(k)
                new_partition[i + 1].extend(partition[i + 1])
                partitions.append(new_partition)
        return partitions

    def free_cumulant(n):
        if n == 0:
            return 0
        return sum(binomial_coefficient(n, k) * (k - 1) ** (n - k) for k in range(1, n + 1)) / fac

    def size(bp):
        return len(bp)
```

```

def trivial_bp(n):
    return [[0] * n for _ in range(n)]

def is_trivial(bp):
    return all(all(x == 0 for x in row) for row in bp)

def transition_matrix(bp, n):
    result = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if bp[i][j] != 0:
                result[i][j] = bp[i][j]
    return result

def l_levy_metric(bp, n):
    tr_matrix = transition_matrix(bp, n)
    cumulant = free_cumulant(n)
    return cumulant

n = random.randint(5, 40)
trivial_bp_value = free_cumulant(n)

if not is_trivial(trivial_bp(n)):
    return {
        "metric_name": "free_cumulant",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "trivial_bp_not_constant"
    }

non_trivial_bps = [random.randint(0, 1) for _ in range(n)] for _ in range(n)]
results = []

for bp in non_trivial_bps:
    l_levy_val = l_levy_metric(bp, n)
    if l_levy_val >= n / 2:
        return {
            "metric_name": "free_cumulant",
            "metric_value": None,
            "instances_tested": len(non_trivial_bps),
            "conjecture_holds": False,
            "counterexample": f"non-trivial BP with  $p(P) \geq n/2$ : {l_levy_val}"
        }
    results.append(l_levy_val)

return {
    "metric_name": "free_cumulant",
    "metric_value": sum(results) / len(results),
    "instances_tested": len(non_trivial_bps),
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys

```

```

seeds = [int(x) for x in sys.argv[1:]] or [random.randint(2, 1000000) for _ in range(30)]

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result["metric_value"])

mean = sum(results) / len(results)
std = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
support_fraction = sum(1 for r in results if r is not None and r >= n / 2) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
elif any(r is not None and r >= n / 2 for r in results):
    first_failing_seed = next(i for i, r in enumerate(results) if r is not None and r >= n / 2)
    print(f"RESULT: FALSIFIED counterexample=\"non-trivial BP with p(P) >= n/2\" first_failing={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE no support for conjecture")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing data; pre-registered support condition cannot be evaluated | next: Run test with extended timeout and debug crash logs

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	116247
2	propose	ollama_remote	qwen3:8b	0	0	50094
3	preregistration	ollama_remote	qwen3:8b	0	0	24804
4	novelty	ollama_remote	qwen3:8b	0	0	20767
5	novelty	ollama_remote	qwen3:8b	0	0	17776
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18447
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11809

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	judge	ollama_remote	qwen3:8b	0	0	18780

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 278724 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/cc2d7e2bed97.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/cc2d7e2bed97.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/cc2d7e2bed97.tar.gz (if generated)